

Recovery Zeta

Méthode de récupération, continuité et anti-répétition d'erreurs pour le projet expérimental hprzeta

Auteur : hprzeta — Date : 26 mai 2026

Ce document rassemble les 20 points de stratégie de récupération pour continuer le travail après plantage, interruption, paquet manquant, consommation excessive de tokens, saturation mémoire, erreur CI, boucle d'agent ou arrêt système.

Principe central : ne jamais relancer immédiatement après un arrêt. On diagnostique, on sauvegarde, on classe l'erreur, on crée un prompt de reprise, puis on relance uniquement la correction minimale et le test ciblé.

1. Problèmes à anticiper sur la machine

Avec un PC ASUS i7, 8 Go RAM, 16 Go swap, GTX 960 4 Go VRAM et Ubuntu 24.04 LTS, les arrêts probables sont : manque de RAM, swap saturé, modèle local trop lourd, Docker/OpenHands trop gourmand, contexte LLM trop long, dépassement de tokens, paquet manquant, conflit de version Python/CUDA/NVIDIA, kernel Jupyter bloqué, calcul zêta trop long, Git sale, interruption manuelle et boucle d'erreur d'agent.

La stratégie recovery doit donc être intégrée au process dès le début, via un agent spécialisé : hprzeta-recovery-agent.

2. Principe général : checkpoint permanent

Chaque action importante doit créer un checkpoint. Le checkpoint permet de reprendre depuis le dernier état stable sans repartir de zéro.

À sauvegarder automatiquement : tâche en cours, prompt utilisé, modèle utilisé, contexte envoyé, fichiers modifiés, git diff, logs terminal, erreurs, paquets installés, tests lancés, consommation tokens approximative, état RAM/swap, prochaine action recommandée.

3. Dossier spécial de récupération

Ajouter un dossier recovery hors dépôt principal pour conserver logs, crash reports, prompts de reprise, mémoire d'erreurs et files de récupération.

```
~/hprzeta-lab/  
  ll_recovery/  
    checkpoints/  
    crash_reports/  
    resume_prompts/  
    error_memory/  
    package_fixes/  
    token_logs/  
    system_snapshots/  
    failed_attempts/  
    recovery_queue/  
  
hprzeta/  
  .agents/  
    recovery/  
      recovery_policy.md  
      error_taxonomy.md  
      resume_template.md  
      package_fix_protocol.md  
      token_budget_policy.md  
      crash_triage_prompt.md
```

4. Agent spécialisé : rôle et responsabilités

Nom recommandé : hprzeta-recovery-agent ou hprzeta-continuity-agent.

Mission : surveiller les interruptions, diagnostiquer la cause, sauvegarder l'état, proposer une reprise sûre, empêcher la répétition de l'erreur et mettre à jour la mémoire d'erreurs du projet.

Déclencheurs : test échoué, commande plantée, RAM faible, modèle trop lourd, budget tokens dépassé, paquet manquant, PR/CI échouée, calcul trop long, agent qui répète la même correction, session IA interrompue.

5. Politique de récupération en 7 étapes

Étape 1 — Stopper proprement

Ne pas relancer immédiatement. Sauvegarder les logs, git diff, état système, puis classer l'erreur.

```
TS=$(date +%F_%H-%M-%S)  
DIR=~/.hprzeta-lab/ll_recovery/crash_reports/$TS  
mkdir -p "$DIR"  
free -h > "$DIR/memory.txt"  
swapon --show > "$DIR/swap.txt" || true  
df -h > "$DIR/disk.txt"  
git status > "$DIR/git_status.txt" || true  
git diff > "$DIR/git_diff.patch" || true
```

Étapes 2 à 7

2. Classer l'erreur. 3. Créer un rapport de crash. 4. Mettre à jour la mémoire d'erreurs. 5. Générer un prompt de reprise. 6. Reprendre depuis le dernier checkpoint. 7. Vérifier que l'erreur ne se répète pas.

6. Taxonomie des erreurs

Créer `agents/recovery/error_taxonomy.md` avec les codes suivants.

```
E001_PACKAGE_MISSING
E002_VERSION_CONFLICT
E003_MEMORY_RAM
E004_SWAP_OVERUSE
E005_VRAM_LIMIT
E006_TOKEN_LIMIT
E007_CONTEXT_TOO_LONG
E008_TEST_FAILURE
E009_GIT_DIRTY_STATE
E010_DOCKER_FAILURE
E011_MODEL_TOO_LARGE
E012_TIMEOUT
E013_NOTEBOOK_KERNEL_CRASH
E014_CI_FAILURE
E015_REPEATED_AGENT_LOOP
E016_NUMERICAL_PRECISION_FAILURE
E017_REFERENCE_MISMATCH
E018_USER_INTERRUPT
```

Chaque erreur doit avoir : cause probable, commande fautive, fichier concerné, correction appliquée, résultat correction et règle pour ne pas répéter.

7. Rapport de crash JSON

Chaque crash doit produire un fichier JSON exploitable par l'agent de reprise.

```
{
  "crash_id": "2026-05-26_15-42-10_E003_MEMORY_RAM",
  "project": "hprzeta",
  "branch": "feature/first-zeros",
  "agent": "hprzeta-coder-local",
  "model": "qwen2.5-coder:3b",
  "error_type": "E003_MEMORY_RAM",
  "command": "pytest -q tests/integration",
  "last_successful_step": "unit tests passed",
  "failed_step": "integration tests",
  "memory_before": "7.6G used / 8G",
  "swap_before": "10G used / 16G",
  "git_dirty": true,
  "files_modified": ["src/hprzeta/zeta/hardy.py", "tests/test_hardy.py"],
  "recommended_resume": "run only targeted test; reduce model context; close browser",
  "do_not_repeat": ["do not run full integration tests with Ollama 7B loaded"]
}
```

8. Mémoire d'erreurs

Créer `~/hprzeta-lab/11_recovery/error_memory/error_memory.jsonl`. Cette mémoire doit être consultée avant chaque nouvelle action.

```
{
  "date": "2026-05-26",
  "error": "E006_TOKEN_LIMIT",
  "cause": "prompt too large",
  "fix": "summarized context and used checkpoint",
  "avoid": "do not resend full report; use RAG chunks only"
},
{
  "date": "2026-05-26",
  "error": "E003_MEMORY_RAM",
  "cause": "OpenHands + Ollama 7B + browser",
  "fix": "closed browser and used qwen2.5-coder:1.5b",
  "avoid": "one model and one agent at a time"
}
```

9. Prompt de reprise

Le prompt de reprise doit être généré automatiquement dans `~/hprzeta-lab/11_recovery/resume_prompts/`.

```
# Prompt de reprise hprzeta

Tu es hprzeta-recovery-agent.
Nous reprenons après interruption.

## Dernier état stable
- Branche : feature/first-zeros
- Dernier commit : abc123
- Dernier test OK : tests/unit/test_theta.py
- Dernier fichier modifié : src/hprzeta/zeta/hardy.py

## Erreur rencontrée
- Type : E006_TOKEN_LIMIT
- Cause : contexte trop long envoyé au modèle
- Modèle utilisé : qwen2.5-coder:3b

## Règles de reprise
1. Ne pas relire tout le dépôt.
2. Lire uniquement README_LOCAL.md, issue courante, git diff, dernier crash report.
3. Résumer avant d'agir.
4. Proposer une seule correction.
5. Lancer uniquement le test ciblé.
6. Générer un rapport court.
```

10. Reprise depuis le dernier checkpoint

L'agent lit dans l'ordre : `latest_checkpoint.json` ou `latest_checkpoint_path.txt`, `latest_crash_report.json` ou `latest_crash_path.txt`, `error_memory.jsonl`, git status, git diff, issue courante, tests échoués. Il agit seulement après ce diagnostic.

11. Vérification anti-répétition

Après correction, relancer uniquement le test minimal, vérifier que le même code erreur n'apparaît plus, écrire `recovery_success`, mettre à jour `error_memory` et créer un mini rapport.

12. Gestion des paquets manquants

En cas de `ModuleNotFoundError`, `ImportError`, `command not found` ou `package not found`, l'agent doit identifier le paquet, vérifier son type, proposer une commande, mettre à jour les fichiers d'environnement, installer, relancer le test ciblé et documenter.

```
# Python
pip install nom_du_paquet
pip freeze > ~/hprzeta-lab/01_env/pip_freeze_$(date +%F).txt
pip freeze | grep nom_du_paquet >> requirements-minimal.txt

# Ubuntu
sudo apt install nom-paquet
dpkg -l | grep nom-paquet >> ~/hprzeta-lab/01_env/apt_installed.log
```

Interdiction : installer un gros requirements complet sans diagnostic minimal.

13. Gestion des interruptions par tokens

Créer `.agents/recovery/token_budget_policy.md`. Les budgets locaux recommandés sont : planification simple 2 000 à 4 000 tokens, correction ciblée 4 000 à 8 000 tokens, résumé rapport 4 000 tokens, RAG maximum 5 chunks, et interdiction de coller tout le PDF ou tout le dépôt dans le prompt.

En cas de dépassement : arrêter la génération, résumer le contexte en 20 lignes, sauvegarder ce résumé, relancer avec résumé + fichiers ciblés, réduire RAG de 5 à 3 chunks et réduire le contexte modèle à 4096.

```
Tu dois travailler avec un budget contexte strict.
Ne lis pas tout le dépôt.
Ne répète pas le rapport complet.
Utilise uniquement : résumé checkpoint, git diff, fichier concerné, test échoué, maximum 3 extraits RAG.
Si l'information manque, demande une récupération ciblée.
```

14. Gestion des boucles d'erreurs

Définir `MAX_RETRY_SAME_ERROR = 2`. Après deux échecs identiques : stop, classer `E015_REPEATED_AGENT_LOOP`, interdire nouvelle tentative automatique, demander un diagnostic différent ou passer au superviseur.

```
{
  "error": "E015_REPEATED_AGENT_LOOP",
  "same_error_count": 3,
  "action": "stop_auto_retry",
  "next_agent": "hprzeta-supervisor",
  "required": "human review or alternative method"
}
```

15. Gestion Git pour reprise propre

Chaque session IA doit travailler sur une branche dédiée. Jamais de modification directe sur main, jamais de git reset --hard automatique, jamais de suppression sans sauvegarde patch.

```
git checkout -b feature/nom-tache

git status
git diff > ~/hprzeta-lab/11_recovery/checkpoints/pre_action.patch

git add .
git commit -m "checkpoint: stable step before recovery risk"

# Si plantage
git status
git diff
git stash push -m "recovery-stash-$(date +%F_%H-%M)"
```

16. Script checkpoint automatique

Créer `scripts/checkpoint.sh`.

```
#!/usr/bin/env bash
set -e
TS=$(date +%F_%H-%M-%S)
DIR="$HOME/hprzeta-lab/11_recovery/checkpoints/$TS"
mkdir -p "$DIR"

pwd > "$DIR/pwd.txt"
git branch --show-current > "$DIR/git_branch.txt" || true
git rev-parse HEAD > "$DIR/git_commit.txt" || true
git status > "$DIR/git_status.txt" || true
git diff > "$DIR/git_diff.patch" || true
free -h > "$DIR/memory.txt"
swapon --show > "$DIR/swap.txt" || true
df -h > "$DIR/disk.txt"
python --version > "$DIR/python_version.txt" 2>&1 || true
pip freeze > "$DIR/pip_freeze.txt" 2>&1 || true

echo "$DIR" > "$HOME/hprzeta-lab/11_recovery/latest_checkpoint_path.txt"
echo "Checkpoint saved: $DIR"
```

17. Script diagnostic après crash

Créer scripts/diagnose_crash.sh.

```
#!/usr/bin/env bash
TS=$(date +%F-%H-%M-%S)
DIR="$HOME/hprzeta-lab/11_recovery/crash_reports/$TS"
mkdir -p "$DIR"

echo "Crash diagnostic $TS" | tee "$DIR/summary.txt"
free -h | tee "$DIR/free.txt"
swapon --show | tee "$DIR/swap.txt" || true
df -h | tee "$DIR/disk.txt"
git status | tee "$DIR/git_status.txt" || true
git log --oneline -5 | tee "$DIR/git_log.txt" || true
python --version | tee "$DIR/python.txt" || true
pip freeze > "$DIR/pip_freeze.txt" 2>&1 || true

echo "$DIR" > "$HOME/hprzeta-lab/11_recovery/latest_crash_path.txt"
echo "Crash report saved: $DIR"
```

18. Prompt complet hprzeta-recovery-agent

Tu es hprzeta-recovery-agent.

Tu intervies après un arrêt, plantage, erreur de paquet, dépassement mémoire, dépassement token, crash notebook, erreur CI ou boucle d'agent.

Objectif : reprendre le travail sans repartir de zéro et sans reproduire l'erreur.

Procédure :

1. Lire latest_checkpoint_path.txt.
2. Lire latest_crash_path.txt si disponible.
3. Lire error_memory.jsonl.
4. Classer l'erreur selon error_taxonomy.md.
5. Identifier la dernière étape stable.
6. Identifier la première étape échouée.
7. Proposer une correction minimale.
8. Interdire la répétition de la cause.
9. Générer un resume prompt.
10. Relancer uniquement le test ciblé.
11. Écrire recovery_success ou recovery_failed.

Contraintes : pas de gros modèle, pas de lecture complète du dépôt, pas de git reset --hard, pas d'installation non journalisée, RAG max 3 extraits, jamais de prétention de preuve RH.

Format : Diagnostic, Cause probable, Dernier état stable, Correction minimale, Commandes proposées, Tests à relancer, Risque de répétition, Règle ajoutée à error_memory, Prompt de reprise.

19. Workflow Recovery global

```
ERREUR / INTERRUPTION
↓
scripts/diagnose_crash.sh
↓
hprzeta-recovery-agent
↓
classification erreur
↓
sauvegarde git diff + logs
↓
mise à jour error_memory
↓
création resume_prompt
↓
correction minimale
↓
test ciblé
↓
rapport recovery
↓
reprise normale
```

20. Exemples et politique finale

Exemples typiques

Paquet manquant : classer E001_PACKAGE_MISSING, installer minimalement, mettre à jour requirements, relancer uniquement le test ciblé.

Trop de tokens : classer E006_TOKEN_LIMIT / E007_CONTEXT_TOO_LONG, résumer checkpoint, limiter RAG à 3 chunks, relancer avec contexte 4096.

RAM saturée : classer E003_MEMORY_RAM / E004_SWAP_OVERUSE, arrêter modèle lourd, fermer navigateur, passer de qwen2.5-coder:3b à 1.5b, reporter OpenHands.

Test numérique échoué : classer E016_NUMERICAL_PRECISION_FAILURE ou E017_REFERENCE_MISMATCH, vérifier précision, formule theta, référence LMFDDB, augmenter précision localement.

Recovery policy prête à copier

```
# hprzeta Recovery Policy

## Objectif
Garantir la continuité du travail après plantage, interruption, erreur de paquet, dépassement mémoire, dépassement token, crash notebook, erreur CI ou boucle d'agent.
```

```
## Règles absolues
1. Ne jamais relancer sans diagnostic.
2. Ne jamais repartir de zéro si un checkpoint existe.
3. Ne jamais répéter deux fois la même erreur.
4. Ne jamais installer un paquet sans journaliser.
5. Ne jamais utiliser un gros modèle si l'erreur est mémoire/tokens.
6. Ne jamais faire git reset --hard automatiquement.
7. Toujours générer un resume prompt.
8. Toujours écrire un rapport de récupération.

## Procédure
Lancer diagnose_crash.sh, lire checkpoints et crash reports, lire error_memory, classer l'erreur, identifier la
dernière étape stable, proposer correction minimale, relancer test ciblé, écrire recovery report, mettre à jour
error_memory.

## Stratégies de repli
- RAM saturée : modèle 1.5B, fermer Docker/OpenHands/navigateur.
- Tokens dépassés : résumé checkpoint, RAG max 3 chunks, contexte 4096.
- Paquet manquant : installer minimalement, mettre à jour requirements.
- Test échoue : relancer test ciblé, pas suite complète.
- Boucle agent : stop après 2 tentatives et demander superviseur.
```

Résumé opérationnel

À chaque arrêt :

```
bash scripts/diagnose_crash.sh
bash scripts/checkpoint.sh
```

Puis donner à l'IA :
Lis latest_checkpoint_path.txt, latest_crash_path.txt et error_memory.jsonl.
Classe l'erreur.
Ne recommence pas la même action.
Propose une reprise minimale.
Relance uniquement le test ciblé.
Écris un rapport recovery.

Sources et ancrages techniques utilisés

Ubuntu 24.04 et swap : guides de création de swapfile et stabilisation mémoire [turn5search127, turn5search129].

LLM locaux et limites matérielles : OpenHands local LLM, Ollama et contraintes RAM/VRAM pour petits modèles [turn5search118, turn5search115, turn5search116].

Référence numérique zêta : LMFDB pour les zéros de $\zeta(s)$, et travaux Platt pour le calcul massif et la vérification [turn1search1, turn1search5, turn1search6, turn1search22].

Connecteurs et outillage : filesystem MCP et GitHub MCP pour état fichiers, issues, PR et logs CI [turn3search105, turn3search106, turn3search108].